

Smart Guardian:

자율주행 로봇 Limo 활용 기반 스마트 관제 시스템

자율주행 로봇 Limo를 활용한 실시간 감시와 데이터 기반 관제 시스템을 구현한 프로젝트입니다. 스마트 시티 환경에서 자율 순찰, 화재 감지, 객체 추적 및 데이터 기반 관제 기능을 통합하여 효율적인 모니터링 솔루션을 제공합니다.



Python



ROS



YOLOv4



OpenCV



FastAPI



팀 소개 및 역할



이한솔 (팀장)

- ✓ 프로젝트 총괄
- ✓ FSM(유한 상태 기계) 기반 자율 순찰 시스템 구현

ROS

SLAM

FSM



정용태

- ✓ 웹 기반 실시간 관제 시스템 구현
- ✓ FastAPI, 웹 대시보드, ros-bridge 통신

FastAPI

Web



맹진수

- ✓ 객체 인식 및 데이터 수집/시각화 시스템 구현
- ✓ YOLOv4 기반 객체 인식 노드, 로그 기록 및 시각화

YOLOv4

OpenCV



최용규

- ✓ Aruco 마커 기반 인식 및 제어 기능 구상 향후 발전 과제
- ✓ 향후 발전 과제

AruCo

Future Work

프로젝트 개요 및 기술 스택

🎯 프로젝트 목표



자율 순찰

SLAM 기술을 이용한 자율적 지역 순찰 수행



화재 감지 및 객체 추적

카메라를 통해 화재 및 사람, 차량 등 객체 실시간 감지



데이터 기반의 스마트 모니터링

순찰 데이터 분석을 통해 유동 인구 패턴 파악 및 스마트 관제



활용 플랫폼

wego Limo, Ubuntu(20.04), ROS1(Noetic)

📦 주요 기술 스택



ROS1 (Noetic)

로봇 하드웨어와 소프트웨어 모듈 간 통신 및 통합



Python

FSM, 객체 감지, 웹 서버 등 주요 로직 구현



SLAM

LiDAR 센서 데이터를 이용한 지도 생성 및 위치 추정



YOLOv4

카메라 영상을 기반으로 객체 탐지 및 추적



FastAPI

웹 기반 실시간 관제 대시보드 및 원격 제어를 위한 백엔드 API 서버

프로젝트 목표 및 시스템 아키텍처

프로젝트의 3가지 핵심 목표

자율 순찰 및 원격 관제

SLAM으로 생성된 지도를 기반으로 로봇이 자율적으로 순찰하며, 관리자는 웹 대시보드를 통해 원격으로 로봇을 제어할 수 있습니다.

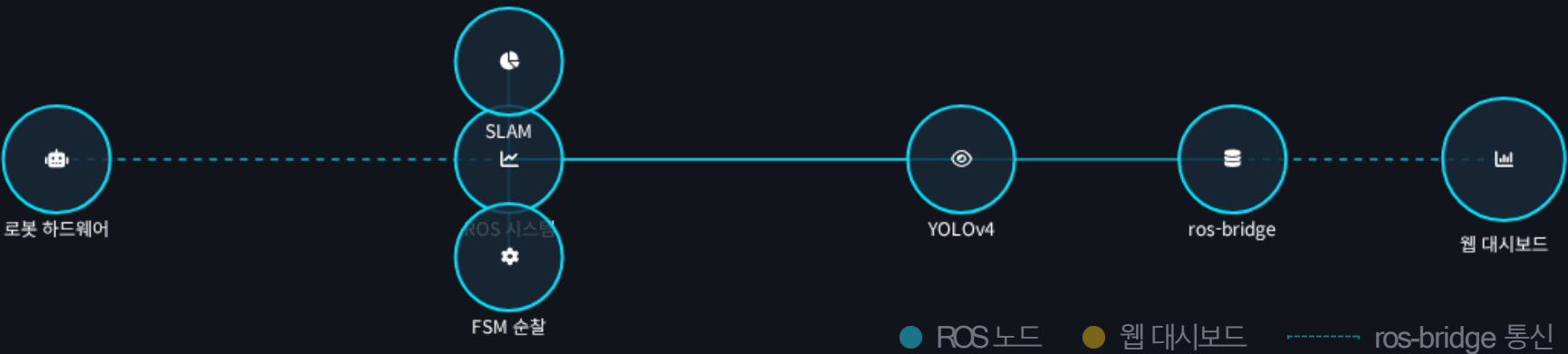
실시간 객체 인식 및 상황 감지

로봇의 카메라를 통해 실시간으로 화재를 감지하고, YOLOv4를 이용해 사람과 같은 객체를 인식하여 비상 상황에 대응합니다.

데이터 기반의 스마트 모니터링

순찰 중 감지된 객체 데이터를 기록하고 분석하여, 유동 인구 패턴 파악 등 스마트한 관제를 지원합니다.

시스템 아키텍처



자율주행의 핵심 기술: SLAM

SLAM이란?

SLAM(Simultaneous Localization and Mapping)은 **동시 위치 추정 및 지도 생성** 기술입니다.



로봇이 **미지 환경을 탐색**하면서 **현재 위치를 추정**하고, 이를 통해 자율적으로 순찰할 수 있습니다.

Smart Guardian에서의 SLAM

- ✓ LiDAR 센서 데이터를 이용해 실시간으로 지도를 생성
- ✓ 로봇의 현재 위치를 지속적으로 추정
- ✓ ROS 시스템 내에서 SLAM 노드는 위치 정보를 다른 노드들과 공유
- ✓ FSM 순찰 시스템이 위치 정보를 기반으로 결정을 내림

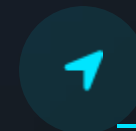
SLAM 작동 원리



센서 데이터
수집



지도
생성



위치
추정



경로
계획



📌 SLAM은 로봇의 "뇌" 역할을 하며, 주요 장소들을 기억하고 위치를 판단하는 기능을 담당합니다.

patrol_limo 노드: if-else 기반 로직의 한계

초기순찰로직:복잡한if-else구문

```
// 순찰상태검사
if (robot_state == "patrolling") {
    // 장애물검사
    if (obstacle_detected) {
        // 회피로직
        if (obstacle_distance < MIN_DIST) {
            if (left_clear) {
                turn_left();
            } else if (right_clear) {
                turn_right();
            } else {
                move_backward();
            }
        }
    }
}
// 목적지도착검사
if (goal_reached) {
    if (current_goal < goals.size() - 1) {
        current_goal++;
        set_next_goal();
    } else {
        patrol_complete = true;
    }
}
// 정체 검사
if (time_since_last_move > STUCK_THRESHOLD) {
    if (distance_since_last_move < MIN_MOVE_DIST) {
        robot_stuck = true;
    }
}
// 정체 복구 시도
if (robot_stuck) {
    move_backward();
    if (time_since_stuck_recovery > RECOVERY_TIME) {
        stop_robot();
        robot_stuck = false;
    }
}
```

“ 초기 버전의 순찰 로봇은 복잡한 *if-else* 조건문으로 모든 동작을 구현했습니다. 이 방식은 코드의 가독성과 유지보수성을 급격히 저하시키며, 예상치 못한 동작이 발생하기 쉬웠습니다.

if-else 방식의 주요 한계



코드 복잡성 증가

조건이 많아질수록 코드의 가독성과 유지보수성이 급격히 저하
로봇의 상태, 센서 입력, 목적지 도착, 정체 상태 등 다양한 조건을 처리 해야 하므로
코드 구성 복잡



예상치 못한 동작

복잡한 if-else 구조에서 로봇 상태 변화에 따른 예외적인 동작이 발생하기 쉬움
특정 조건의 우선순위나 중첩에 의해 의도하지 않은 결과가 나타날 수 있음.



디버깅 어려움

로봇이 특정 상태에 빠졌거나 예상대로 작동하지 않을 때
해당 상태를 찾아내는 것이 어려움.
복잡한 if-else 구문은 상태 전환 로직을 추적하기 어려움.



확장성 제한

새로운 기능이나 상태를 추가하려면 기존의 if-else 구조를 수정해야 하며,
이는 다시 코드의 복잡성을 증가시키고 오류의 가능성을 높임.

문제점 시연: 단순 로직의 불안정성

if-else 기반 로직의 한계

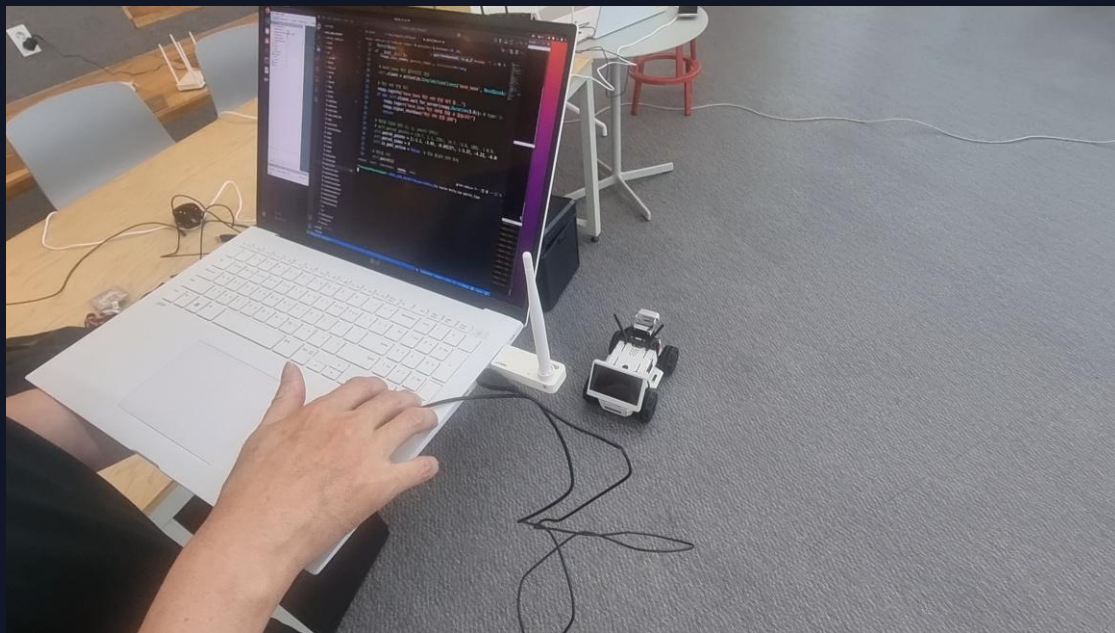
- 조건이 많아질수록 코드의 가독성과 유지보수성이 급격히 저하
- 예상치 못한 동작이 발생하는 등 여러 한계를 드러냄
- 로봇의 상태를 명확히 파악하기 어려움
- 동작의 순서와 흐름을 추적하기 어렵게 만들

⚠ 불안정한 주행 증상

- ❌ 순찰 중간에 멈추는 현상
- ❌ 장애물 근처에서 방향 결정에 애매함
- ❌ 정체 상태에서 자동으로 복구되지 않음

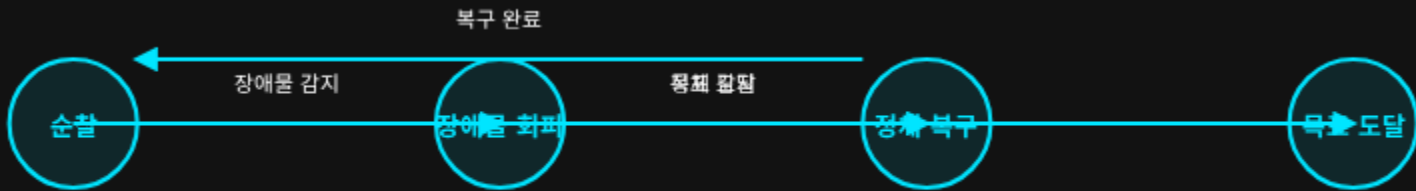
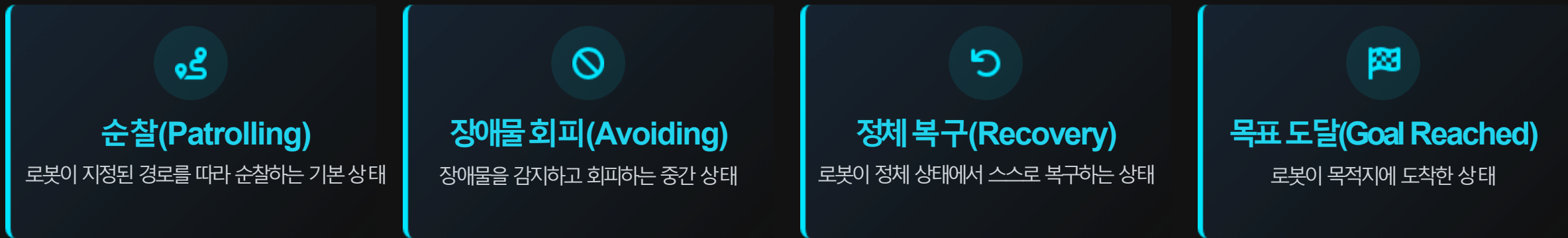
순찰노드 FSM 적용 전 초기버전 영상

■ 시연 영상



FSM 패턴 기반 순찰 시스템 설계

FSM(Finite State Machine, 유한 상태 기계) 패턴은 복잡한 if-else 조건문의 한계를 극복하기 위해 도입되었습니다. 이 패턴은 로봇의 상태를 명확한 단위로 분리하고, 상태 간의 전환을 체계적으로 관리합니다.



- #### </> 코드 가독성

FSM은 복잡한 조건문을 명확한 상태로 분리하여 코드의 가독성을 획기적으로 개선합니다.
- #### 🛡️ 로봇 안정성

정의된 상태와 전환으로 예상치 못한 동작을 방지하고, 시스템의 안정성을 높입니다.
- #### 🔧 시스템 확장성

새로운 상태를 쉽게 추가할 수 있어, 기능 확장 및 유지보수의 용이성을 제공합니다.

FSM의 심장: _update_fsm 함수

💓 FSM의 "심장 박동"

주기적으로 실행되는 메인 루프로, 현재 로봇의 상태를 확인하고 그에 맞는 동작 함수를 호출합니다.

함수의 역할

- ✓ 로봇의 전체 동작 흐름을 명확하고 예측 가능하게 제어
- ✓ 현재 상태에 따라 적절한 실행 함수로 분기
- ✓ 주요 상태: 순찰(Patrolling), 장애물 회피(Avoiding), 정체 복구(Recovery), 목적지 도착(Goal Reached)

FSM 상태 전환

</> _update_fsm 함수 코드

```
def _update_fsm(self, event):  
  
    """  
    FSM의 '심장 박동(heartbeat)'과 같은 메인 루프.  
    주기적으로 호출되며, 현재 상태에 따라 적절한 실행 함수로 분기합니다.  
    """  
  
    if self.current_state == self.STATE_PATROLLING:  
        self._execute_patrolling()  
    elif self.current_state == self.STATE_AVOIDING:  
        self._execute_avoiding()  
    elif self.current_state == self.STATE_RECOVERY:  
        self._execute_recovery()  
    elif self.current_state == self.STATE_GOAL_REACHED:  
        self._execute_goal_reached()
```

FSM 동작 파라미터 튜닝



FSM의 유연성을 극대화하기 위해, 로봇의 핵심 동작 파라미터들을 별도의 **YAML** 설정 파일로 분리했습니다.
이를 통해 코드를 직접 수정하지 않고도 로봇의 세부 동작을 현장 환경에 맞게 손쉽게 최적화할 수 있습니다.

patrol_fsm_params.yaml

=== ★ 핵심 튜닝 파라미터: 정체 (Stuck) 감지 ===

stuck_threshold_time: 10.0 # [★매우 중요] 이 시간(초) 동안 거의 움직임이 없으면 '정체'로 판단

stuck_threshold_dist: 0.1 # [★중요] stuck_threshold_time 동안 이 거리(m) 미만으로 움직였을 경우 정체로 판단

=== 복구 동작 관련 파라미터 ===

recovery_backup_speed: -0.1 # 정체 복구 시 후진 속도 (m/s)

recovery_backup_time: 0.5 # 정체 복구 시 후진하는 시간 (초)

move_base의 한계: 정체 문제

move_base란?

ROS의 `move_base`은 강력한 경로 계획 패키지로, 로봇에게 목적지까지의 최적 경로를 계산하고 제어합니다.

move_base의 한계



좁은 공간

경로 계획이 불가능한 좁은 공간에서 정체 문제 발생



복잡한 장애물

복잡한 환경에서 경로를 찾지 못하고 멈추는 현상

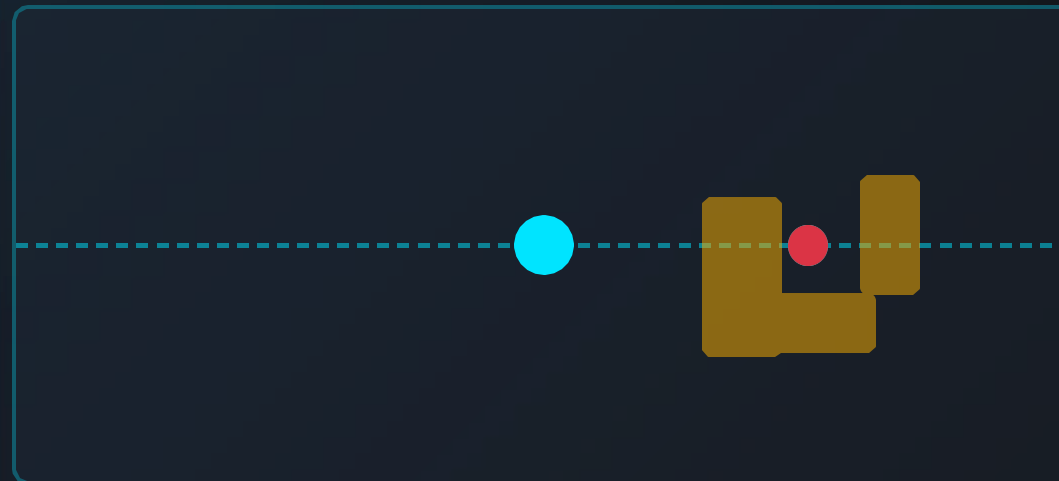


로봇 위치 오류

로봇의 실제 위치와 SLAM 지도상의 위치 불일치로 인한 오작동



정체 문제 시각화



● 로봇

● 장애물

--- 계획된 경로

● 정체 위치



문제 해결 방안

FSM(Finite State Machine) 패턴을 도입하여 로봇이 정체 상태를 감지하고, 스스로 해결하도록 하는 **RECOVERY** 상태 추가

→ 다음 슬라이드에서 **RECOVERY** 상태에 대해 자세히 설명

FSM의 RECOVERY 상태: _execute_recovery 함수

RECOVERY 상태 목적

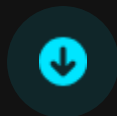
ROS의 move_base은 좁은 공간이나 복잡한 장애물 환경에서 경로를 찾지 못하고 멈추는 '정체' 문제가 발생할 수 있습니다.

FSM에 RECOVERY 상태를 추가하여 이 문제를 해결합니다. 로봇이 일정 시간 동안 거의 움직이지 못하는 정체 상태가 감지되면, RECOVERY 상태로 전환됩니다.

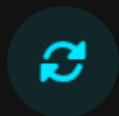
복구 과정



정체 감지



후진 명령



상태 복귀

복구 관련 파라미터

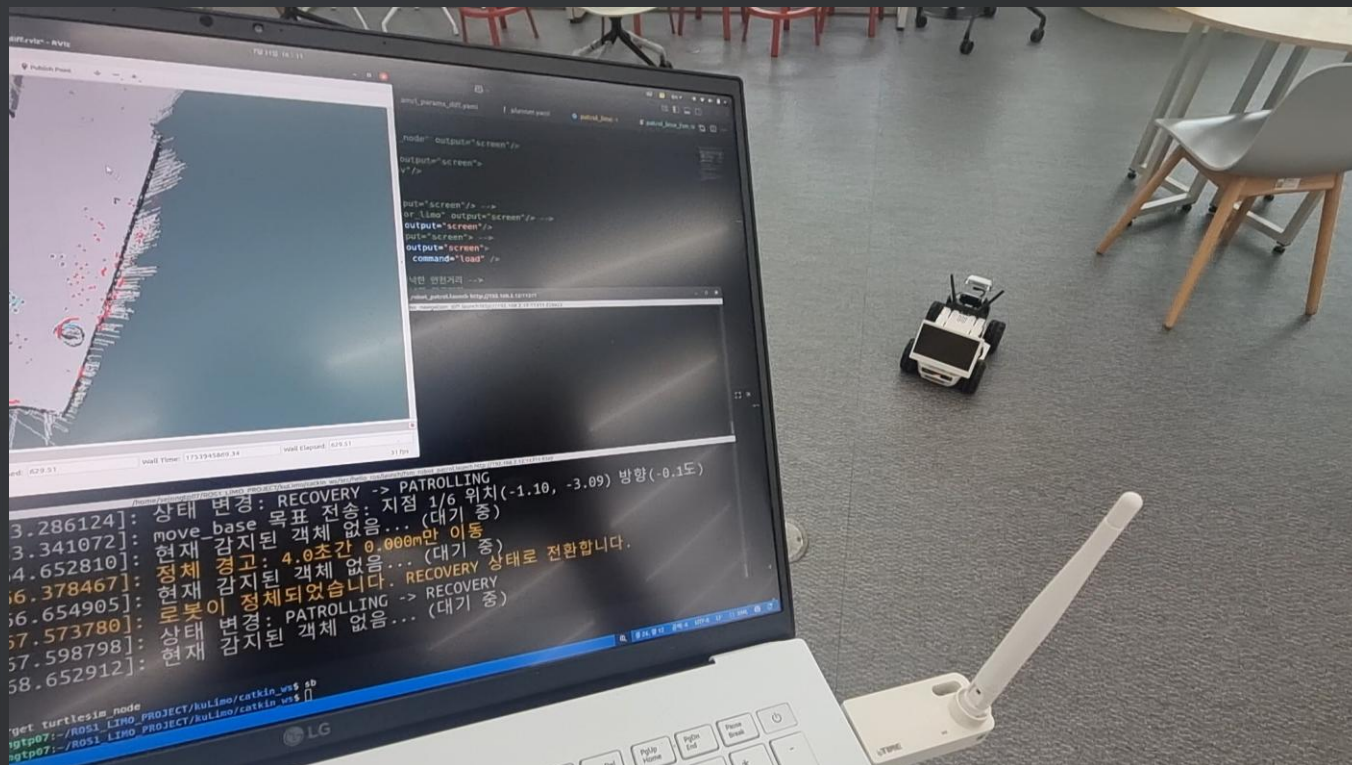
🔧 recovery_backup_speed: -0.1 m/s ⏰ recovery_backup_time: 0.5 초

_execute_recovery 함수 구현

```
def _execute_recovery(self):  
  
    """  
    [RECOVERY 상태]  
    - 역할: cmd_vel을 직접 제어하여 후진함으로써 정체 상황에서 벗어납니다.  
    - 전환 조건: 설정된 시간만큼 후진한 후 PATROLLING 상태로 복귀합니다.  
    """  
  
    # 1. 상태 전환 조건 확인  
    if (rospy.Time.now() - self.last_state_change_time).to_sec() >  
        self.recovery_backup_time:  
        rospy.loginfo("복구 동작 완료. PATROLLING 상태로 복귀합니다.")  
        self._stop_robot()  
        rospy.sleep(0.5)  
        self._change_state(self.STATE_PATROLLING) return  
  
    # 2. 현재 상태의 행동 수행  
    twist = Twist()  
    twist.linear.x = self.recovery_backup_speed  
    twist.angular.z = 0.0  
    self.cmd_vel_pub.publish(twist)
```

FSM 기반 순찰 노드 시연

FSM 적용 후 최종 순찰 주행 시연



장애물 회피

로봇이 장애물을 인식하고 자동으로 경로를 수정하여 안전하게 회피합니다.

정체 자가 복구

정체 상태가 감지되면 RECOVERY 상태로 자동 전환되어 스스로 문제를 해결합니다.

복잡한 환경 적응

로봇이 복잡한 상황에서도 안정적으로 순찰 임무를 수행합니다.

파라미터 튜닝

YAML 파일을 통해 FSM 동작 파라미터를 쉽게 조정 가능합니다.

OpenCV 기반 HSV 색상 화재 감지



HSV 색상 공간 변환

카메라로 입력되는 RGB 영상을 HSV(Hue, Saturation, Value) 색상 공간으로 변환합니다. HSV는 색상(Hue) 정보가 조명 변화에 덜 민감하여, 특정 색상(붉은색, 주황색 계열)을 안정적으로 검출하는데 유리합니다.



색상 영역 마스크링 및 윤곽선 분석

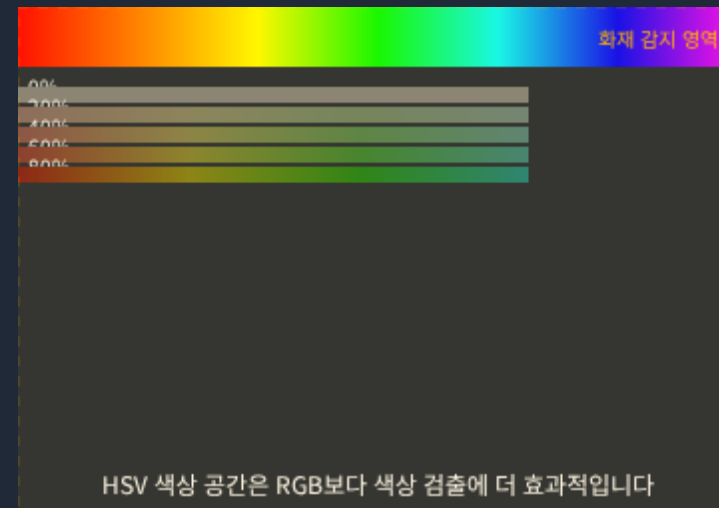
변환된 HSV 영상에서 불꽃의 색상 범위를 지정하여 해당 영역만 추출하는 마스크를 생성합니다. 이후, 마스크 이미지에서 윤곽선(Contour)을 분석하여 불꽃과 유사한 형태의 객체를 최종적으로 감지 합니다.



비상 대응

화재가 감지되면, 로봇은 즉시 순찰 임무를 중단하고 경고음을 발생시켜 관리자에게 위험 상황을 알립니다.

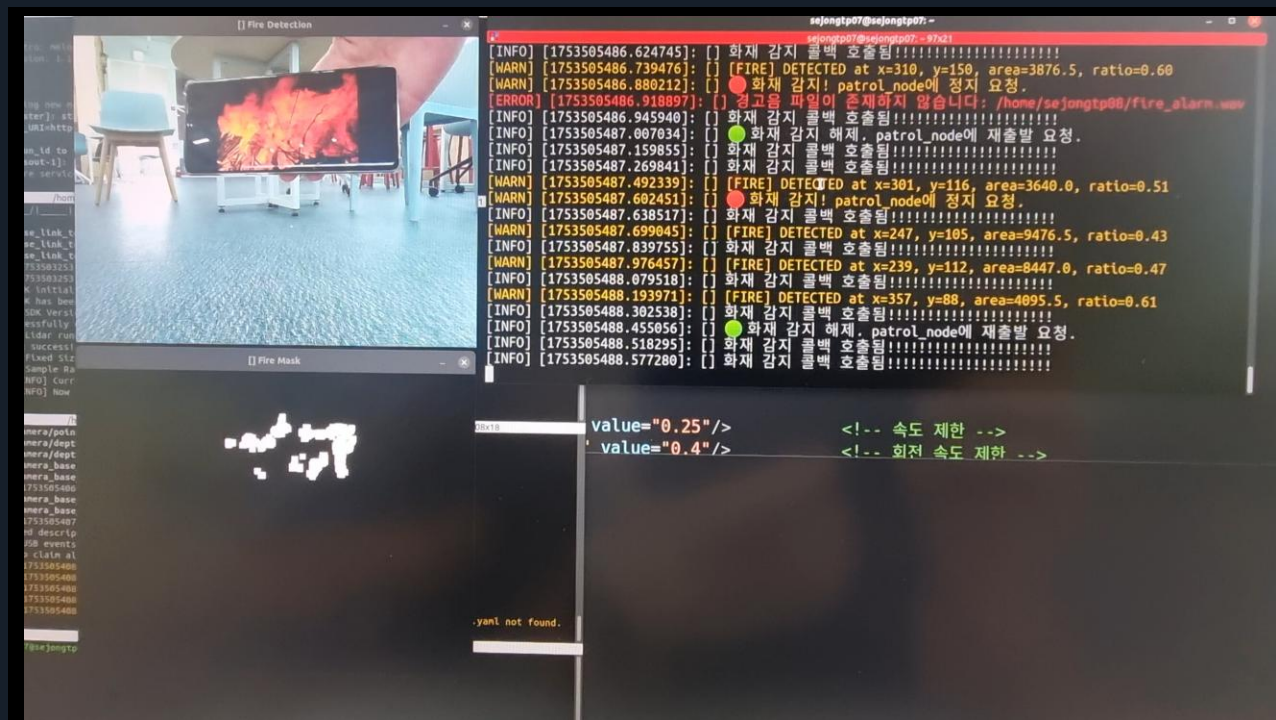
HSV 색상 공간 이해



Hue (색상)	0	180
Saturation (채도)	0	255
Value (명도)	0	255

💡 HSV 공간에서 불꽃은 주로 빨간색 또는 주황색으로 나타납니다. 이에 해당하는 Hue 값은 대략 0~30 또는 150~180 정도입니다.

화재 감지 기능 시연



</> 핵심 코드

#OpenCV기반 화재 감지 로직

def detect_fire(self, cv_image): #BGR을 HSV로 변환

hsv = cv2.cvtColor(cv_image, cv2.COLOR_BGR2HSV)

#불꽃의 색상 범위 정의

lower_red = np.array([0, 100, 100])

upper_red = np.array([10, 255, 255])

#마스크 생성 및 화재 감지

mask = cv2.inRange(hsv, lower_red, upper_red) return
cv2.countNonZero(mask) > threshold

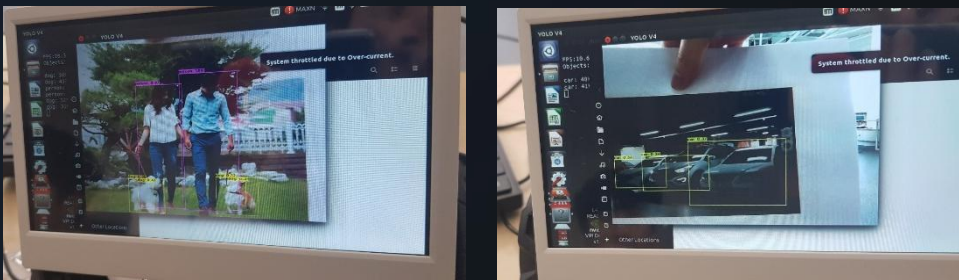
🔥 화재 감지 시스템

- ✓ OpenCV 기반 **HSV 색상 공간** 변환 활용
- ✓ 불꽃의 색상 범위 지정 및 윤곽선 분석
- ✓ 조명 변화에 덜 민감한 안정적 감지

YOLOv4 기반 객체 추적 및 데이터 기록

YOLOv4 객체 인식

- ✔ 실시간 객체 탐지에서 뛰어난 속도와 정확성을 제공하는 YOLOv4 알고리즘 사용
- ✔ 한 번의 이미지 스캔으로 이미지 내 다수 객체의 종류와 위치 동시 파악
- ✔ 로봇 주행 중 카메라를 통해 사전에 정의된 객체 실시간 인식

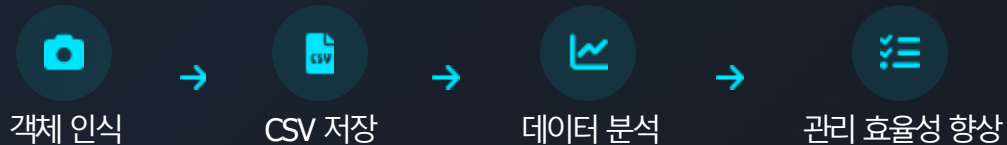


YOLOv4 알고리즘이 적용된 카메라 화면

CSV 데이터 기록 및 분석

- ✔ 인식된 객체의 종류, 시간 등의 정보 CSV 파일 형식으로 자동 기록
- ✔ 데이터는 유동 인구 분석 및 관제 효율성 향상을 위한 기초 자료로 활용

데이터 흐름



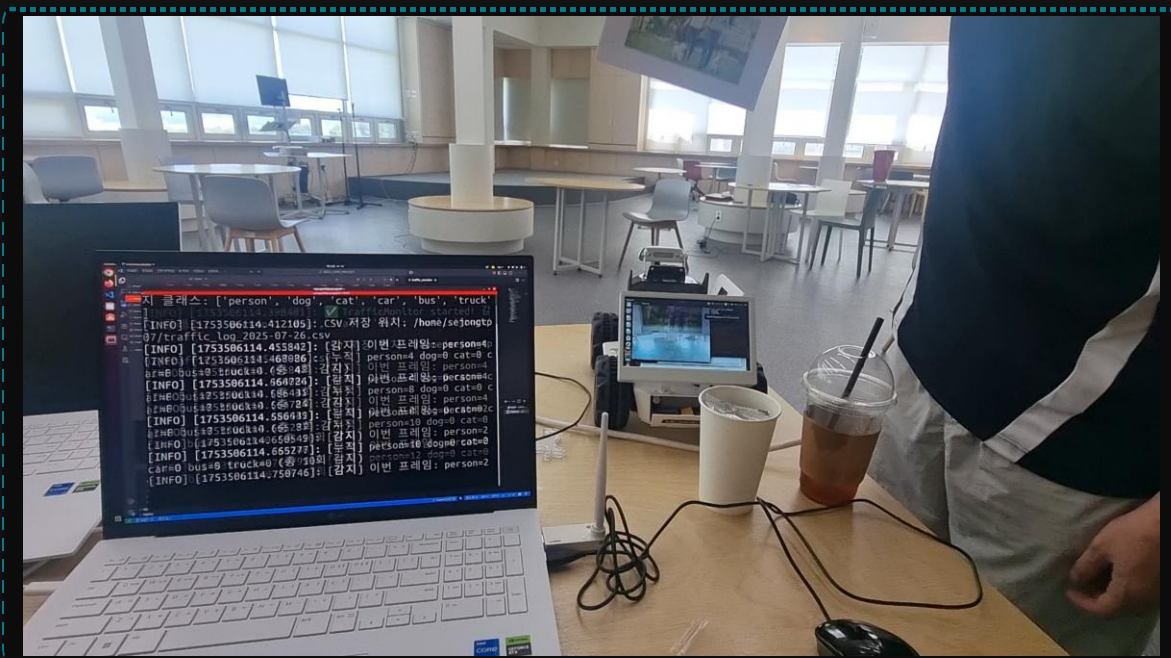
데이터 기반 분석의 가치

시간대별 유동 인구 추세 객체 종류별 비율 특정 구역 특성 파악 순찰 경로 최적화

▶ YOLOv4 객체 인식 시스템 동작 시연 영상을 통해 실제 작동 모습 확인 가능

YOLOv4 객체 추적 시연

YOLOv4 알고리즘은 실시간 객체 탐지에서 뛰어난 속도와 정확성을 보이며, 한 번의 이미지 스캔으로 이미지 내 다수 객체의 종류와 위치를 동시에 파악할 수 있습니다.



실시간 처리

카메라로 입력된 영상에서 즉시 객체를 인식하고, 처리 결과를 화면에 표시합니다.



경계상자 표시

인식된 객체 주변에 경계 상자를 표시하고, 객체의 종류를 클래스 라벨로 함께 출력합니다.



데이터 기록

인식된 객체의 종류, 위치, 시간 등의 정보가 CSV 파일로 자동 기록되어 이후 분석에 활용됩니다.

traffic_monitor 노드: 객체 이동량 분석

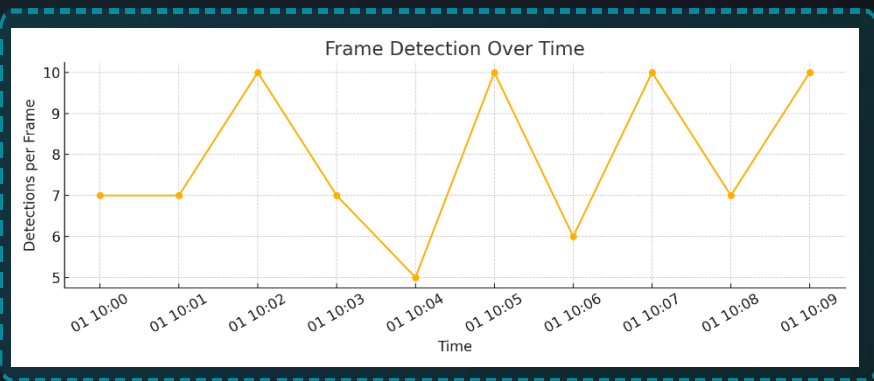
traffic_monitor 노드는 YOLOv4 알고리즘을 통해 수집된 객체 인식 데이터(CSV)를 단순한 기록에 그치지 않고, 관제 효율성을 높이는 중요한 정보로 활용합니다. 이 데이터를 통해 다음과 같은 시각화 자료를 생성하고, 이를 바탕으로 예측 및 예방 중심의 스마트 관제가 가능해 집니다.



시간대별 유동 인구 추세

순찰 경로 최적화

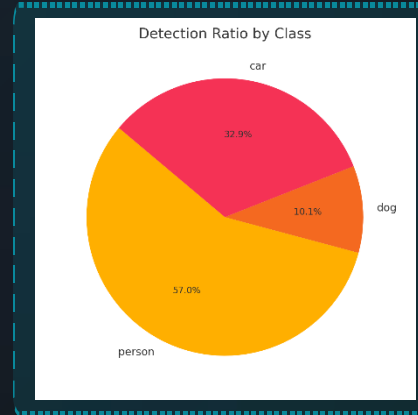
시간별 분석



객체 종류별 비율

구역 특성 파악

객체 분석



활용 가능성

traffic_monitor 노드를 통해 수집된 데이터는 순찰 경로 및 시간 최적화뿐만 아니라, 특정 이벤트의 예측 및 예방에도 활용될 수 있습니다. 예를 들어, 특정 시간에 특정 종류의 객체가 많이 감지되는 패턴이 발견되면, 이를 바탕으로 해당 시간대에 증강된 순찰을 수행하거나, 관리자에게 미리 알림을 보내어 예방적 관리가 가능해 집니다.

웹 대시보드: 실시간 관제 및 원격 제어

FastAPI 기반 웹 서버

FastAPI를 이용한 웹 서버 구현으로, ROS 시스템과 웹 인터페이스 간의 실시간 통신을 가능하게 합니다.

실시간 모니터링

- ✓ 로봇의 현재 위치 및 상태 실시간 확인
- ✓ 카메라 영상 스트리밍 및 객체 감지 정보
- ✓ 모든 시스템 상태의 한눈에 파악

원격 제어

비상 상황이나 특정 임무 수행 시, 관리자가 직접 로봇을 원격으로 조종할 수 있는 기능입니다.

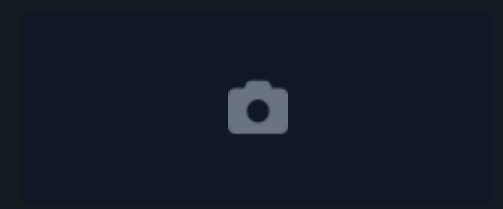
● 로봇 연결 상태: 정상

● 실시간 데이터: 수신 중

Smart Guardian 관제 대시보드

📍 현재 위치: 구역 A

카메라 피드



로봇 상태

배터리	78%
Nhi	정상
속도	0.15 m/s

객체 감지



컨트롤 패널



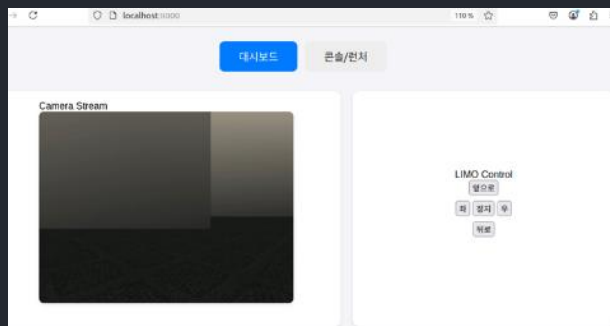
ROS-Bridge

로봇의 ROS 시스템과 웹 서버 간의 실시간 통신을 중개하는 핵심 컴포넌트

웹 대시보드



웹 대시보드 화면 1

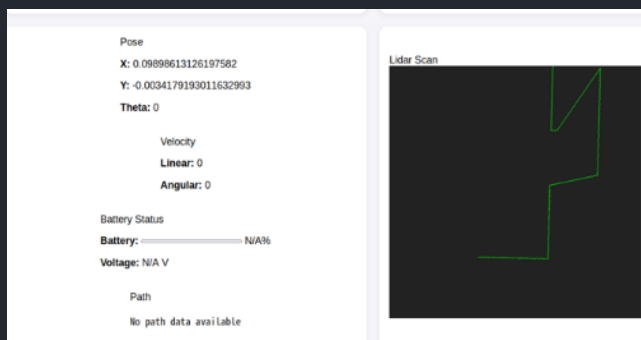


CLI 화면 1

```
[DEBUG] cmd from web: {'direction': 'stop'}
[DEBUG] cmd from web: {'direction': 'left'}
[DEBUG] cmd from web: {'direction': 'stop'}
[DEBUG] cmd from web: {'direction': 'left'}
[DEBUG] cmd from web: {'direction': 'stop'}
[DEBUG] cmd from web: {'direction': 'left'}
[DEBUG] cmd from web: {'direction': 'stop'}
[DEBUG] cmd from web: {'direction': 'left'}
[DEBUG] cmd from web: {'direction': 'stop'}
[DEBUG] cmd from web: {'direction': 'left'}
[DEBUG] cmd from web: {'direction': 'stop'}
[DEBUG] cmd from web: {'direction': 'left'}
[DEBUG] cmd from web: {'direction': 'stop'}
[DEBUG] cmd from web: {'direction': 'left'}
[DEBUG] cmd from web: {'direction': 'stop'}
[DEBUG] cmd from web: {'direction': 'left'}
[DEBUG] cmd from web: {'direction': 'stop'}
```



웹 대시보드 화면 2

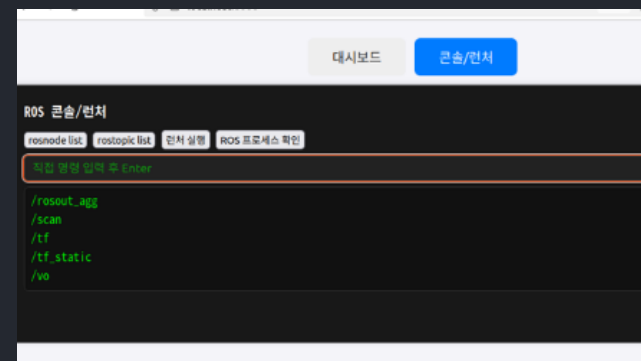


CLI 화면 2

```
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
[DEBUG] pose emit: {'x': 0.09898613126197582, 'y': -0.0034179193011632993, 'thet': 0}
```



웹 대시보드 화면(명령어 입력)



결론 및 향후 발전 방향



프로젝트의 한계

- 장애물 회피 및 정체 상태 극복과 같은 자율주행의 완성도 측면에서는 목표했던 수준에 완벽히 도달하지 못함



향후 발전 방향

이번 프로젝트 경험을 바탕으로, 제한된 하드웨어 환경에서도 더 효율적이고 안정적인 성능을 낼 수 있는 방안을 지속적으로 연구하고 적용해 나갈 것입니다.

순찰 시스템 고도화

딥러닝 기반 경로 예측

동적 경유지 설정

군집 로봇 협업

감지 시스템 향상

딥러닝 화재 감지

열화상 카메라 통합

다중 센서 융합

관제 시스템 확장

실시간 대시보드

모바일 앱 연동

AI 리포트 생성



프로젝트의 성과

- > FSM 기반 안정적인 순찰 시스템 구축
- > 데이터 기반 스마트 분석 시스템
- > 실시간 화재 감지 및 객체 인식 구현
- > 웹 기반 원격 관제 시스템 완성
- > 확장 가능한 아키텍처 설계



문제 해결을 위한 노력

- 다각적 해결책 모색: 제한된 시간과 자원 속에서 웹 검색과 AI를 적극적으로 활용하여 문제 해결 방안을 탐색
- 경로 최적화 시도: A→B 지점 이동 시, 중간에 여러 임시 경유지(Waypoint)를 추가하여 정체 및 회피 문제를 최소화
- 파라미터 튜닝: patrol_fsm_params.yaml 파일의 정체 감지 시간(stuck_threshold_time), 이동 거리(stuck_threshold_dist) 등의 값을 수없이 수정하며 최적의 조합을 찾기 위한 노력

순찰 시스템의 도전 과제와 개선 시도

⚠ 주요 도전 과제

순찰 경로 최적화

- move_base의 정체/타임아웃 빈발
- 복잡한 환경에서의 경로 계획 실패
- 동적 장애물 대응 한계

센서 데이터 처리

- LiDAR 데이터의 노이즈 문제
- 측정 오차로 인한 오탐지
- 환경 변화에 따른 정확도 변동

시스템 안정성

- 예기치 못한 상황에서의 시스템 정지
- 네트워크 통신 지연 및 끊김
- 하드웨어 리소스 제한

✂ 개선 시도 및 해결책

FSM RECOVERY 상태 구현

- move_base 타임아웃 감지 시 자동 전환
- cmd_vel 직접 제어로 후진 동작
- 대안 경로 탐색 및 재시도 로직

동적 경로 재설정

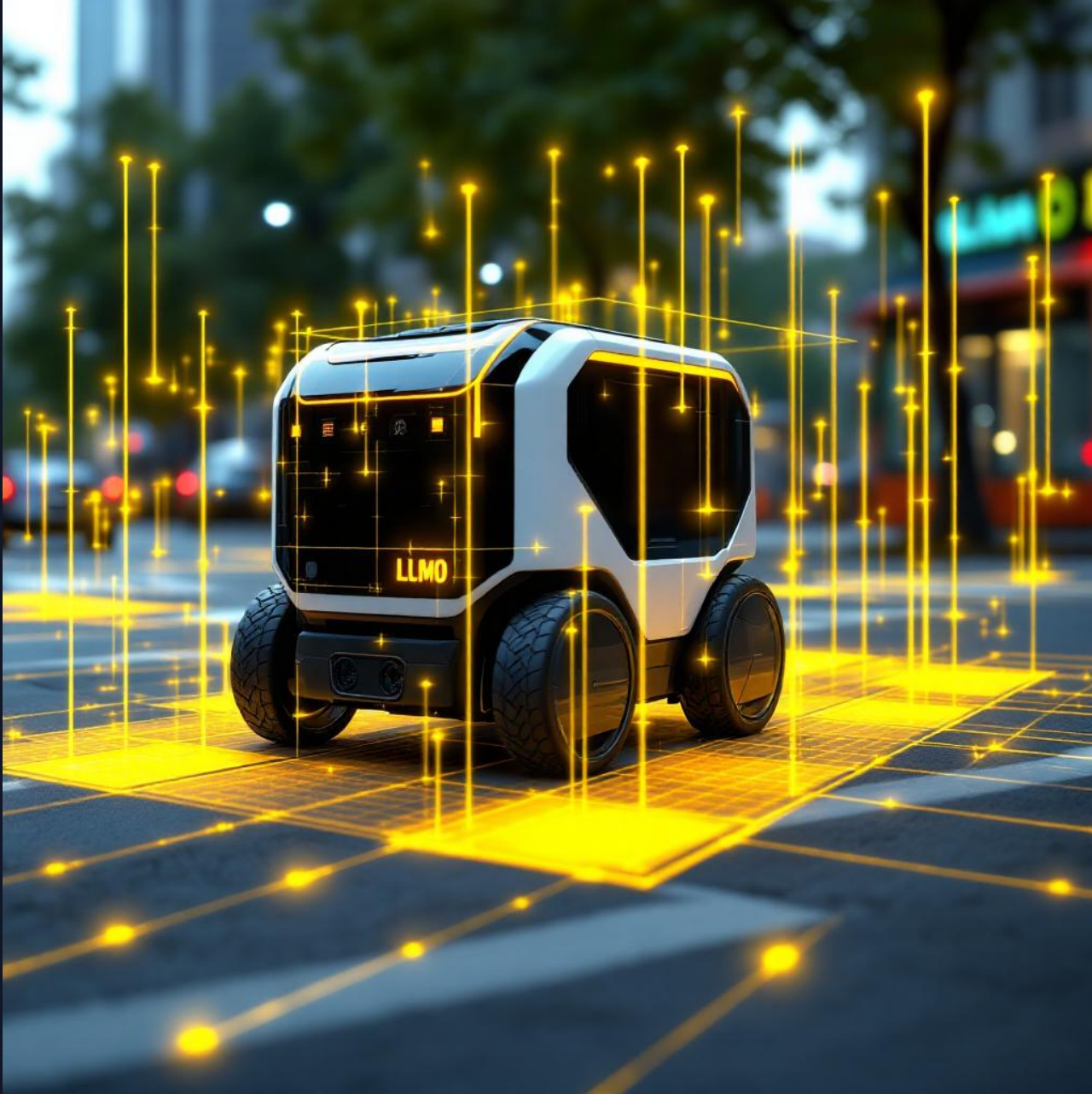
- 실시간 장애물 감지 시 우회 경로 생성
- 순찰 지점 순서 동적 변경
- 환경 적응형 파라미터 자동 조정

모니터링 및 로깅 강화

- 상세한 시스템 상태 로깅
- 실시간 성능 지표 모니터링
- 오류 패턴 분석 및 예방 시스템

💡 핵심 개선 철학

“시스템의 한계를 극복하기 위해 다양한 로직을 설계하고 적용하여 견고하고 신뢰할 수 있는 자율 순찰 시스템을 구축”



Q & A

감사합니다

? 질문이 있으시면 편하게 말씀해주세요